OK, so you've reviewed your code (or got someone else to review it). You're sure there are no bugs. The proof of the pudding is, as they say, in the eating—how can you test your code to confirm or deny your belief in its lack of bugs?

10.2.2. Locating concurrency-related bugs by testing

When developing single-threaded applications, testing your applications is relatively straightforward, if time consuming. You could, in principle, identify all the possible sets of input data (or at least all the interesting cases) and run them through the application. If the application produced the correct behavior and output, you'd know it works for that given set of input. Testing for error states such as the handling of disk-full errors is more complicated than that, but the idea is the same—set up the initial conditions and allow the application to run.

Testing multithreaded code is an order of magnitude harder, because the precise scheduling of the threads is indeterminate and may vary from run to run. Consequently, even if you run the application with the same input data, it might work correctly some times and fail at other times if there's a race condition lurking in the code. Just because there's a potential race condition doesn't mean the code will fail *always*, just that it *might* fail *sometimes*.

Given the inherent difficulty of reproducing concurrency-related bugs, it pays to design your tests carefully. You want each test to run the smallest amount of code that could potentially demonstrate a problem, so that you can best isolate the code that's faulty if the test fails—it's better to test a concurrent queue directly to verify that concurrent pushes and pops work rather than testing it through a whole chunk of code that uses the queue. It can help if you think about how code should be tested when designing it—see the section on designing for testability later in this chapter.

It's also worth eliminating the concurrency from the test in order to verify that the problem is concurrency-related. If you have a problem when everything is running in a single thread, it's just a plain common or garden-variety bug rather than a concurrency-related bug. This is particularly